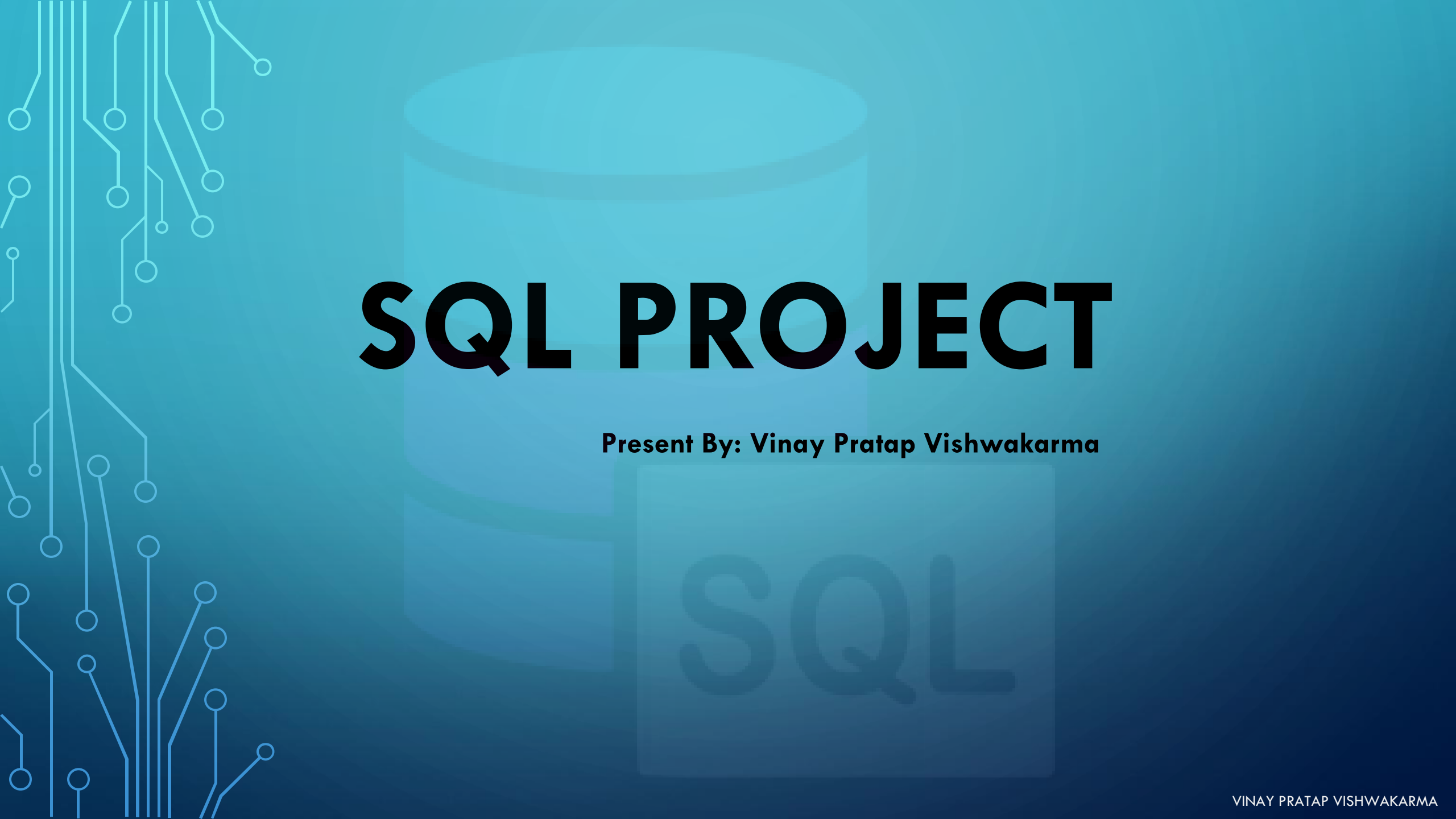


SQL

STRUCTURED QUERY LANGUAGE

LOREM IPSUM DOLOR SIT AMET, CONSECTETUER ADIPISCING ELIT, SED
DIAM NONUMMY NIBH EUISMOD TINCIDUNT UT LAOREET DOLOR





SQL PROJECT

Present By: Vinay Pratap Vishwakarma

OBJECTIVE

We have total 6 table and I Used the provided DDLs to create the following tables:

- a. Customers**
- b. Products**
- c. Orders**
- d. Order Items**
- e. Payments**
- f. Product Review**

OBJECTIVE

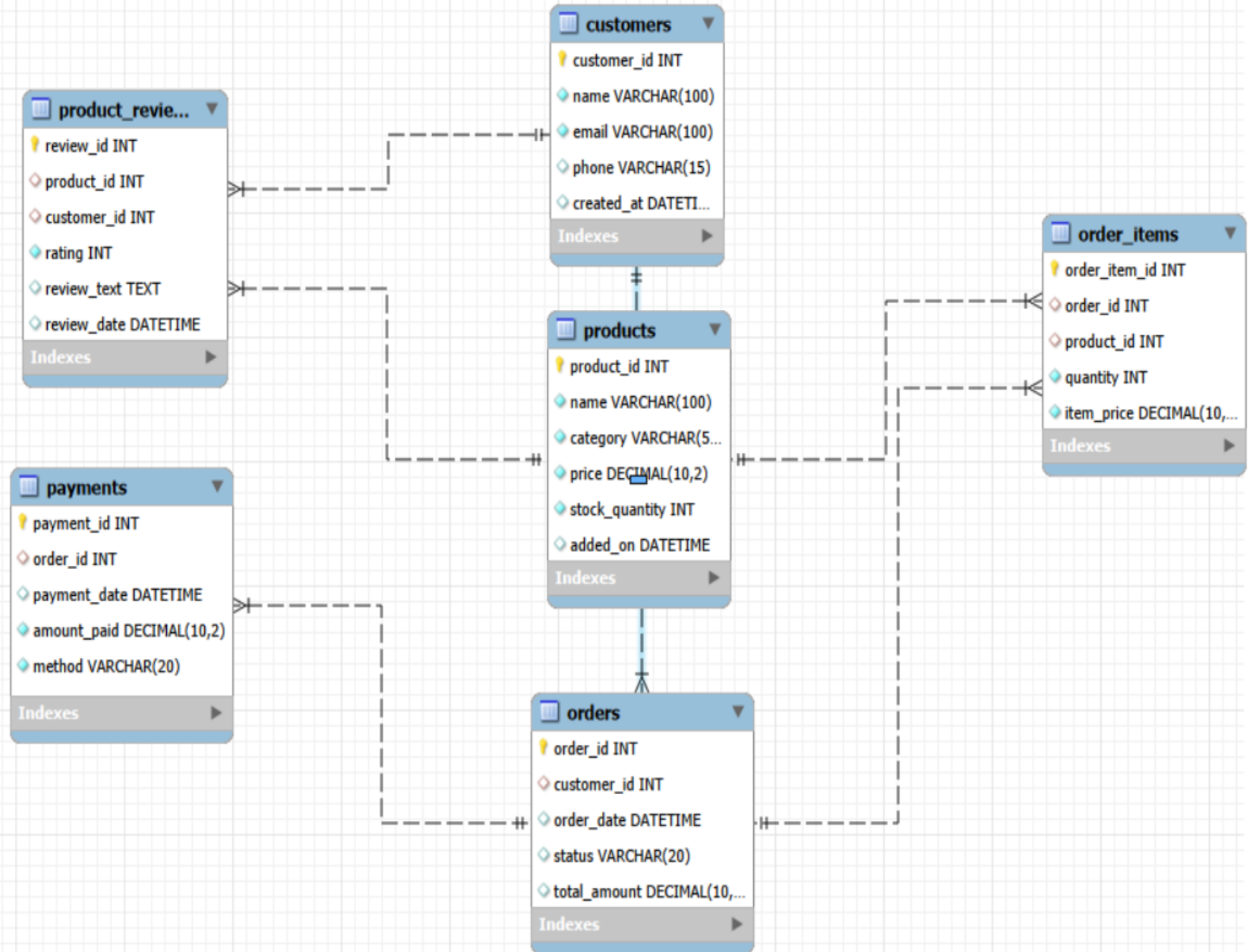
1. **IN THIS PRACTICE WE HAVE A DATABASE OF RETAIL STORE INDUSTRY WHERE I DID SO MANY ANALYSIS WITH THE HELP OF GENERATE BUSINESS INSIGHTS.**
2. **THIS ANALYSIS WILL HELP THE INDUSTRY FOR THEIR VALUABLE DECISION AND ALSO IT WILL SUPPORT THE MANPOWER AND REDUCE TIME BY 30% - 40%.**
3. **I'M TRYING TO KEEP THIS ASSIGNMENT SIMPLE BECAUSE I INCOURAGE THE PEOPLE WHO HAVE ZERO KNOWLEDGE OF TECHNICAL WORD.**

PROBLEM STATEMENT

Retail stores struggle to track customer orders, payments, product sales, and inventory in a centralized system, leading to poor data visibility, inefficient operations, and difficulty in generating accurate business insights

SQL

ER DIAGRAM



The background is a solid blue gradient. On the left, there are white circuit-like lines with circular nodes. In the center, there is a faint illustration of a person with dark hair sitting at a desk with a laptop. Above the person's head, there is a faint box containing the text 'SQL'. To the left of the person, there are faint icons of a database cylinder, a gear, and a hierarchical tree structure.

LEVEL 1

1. Retrieve customer names and emails for email marketing

This helps the marketing team extract basic customer contact details for campaigns.

INPUT

```
SELECT NAME AS CUSTOMERS_NAME, EMAIL AS EMAIL  
FROM CUSTOMERS;
```



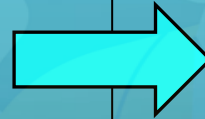
OUTPUT

CUSTOMERS_NAME	EMAIL
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Kaitlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com
Mark Duke	user6@example.com
Briana Wright	user7@example.com
John Bryan	user8@example.com
Jason Thompson	user9@example.com
Shawn Hill	user10@example.com
Walter Jenkins	user11@example.com
Mary Knight	user12@example.com

2. View complete product catalogue with all available details.
The product manager may want to review all product listings in one go.

INPUT

```
SELECT * FROM PRODUCTS;
```



OUTPUT

product_id	name	category	price	stock_quantity	added_on
1	Plant No	Home	639.43	152	2024-01-30 06:30:53
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
9	Common Study	Toys	985.19	171	2023-07-20 13:06:42
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06

3. List all unique product categories

Useful for analysing the range of departments or for creating filters on the website.

INPUT

```
SELECT distinct CATEGORY AS ALL_CATEGORY_LIST  
FROM PRODUCTS;
```



OUTPUT

ALL_CATEGORY_LIST
Home
Clothing
Electronics
Toys
Books

4. Show all products priced above ₹1,000.

This helps identify high-value items for premium promotions or pricing strategy reviews.

INPUT

```
SELECT * FROM products
WHERE price > 1000;
```



OUTPUT

product_id	name	category	price	stock_quantity	added_on
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
13	Full West	Books	2112.33	172	2023-09-15 03:13:38
16	Special Fact	Toys	1094.18	15	2025-03-17 10:42:40

5. Display products within a mid-range price bracket (₹2,000 to ₹5,000).
A merchandising team might need this to create a mid-tier pricing campaign.

INPUT

```
SELECT * FROM products  
WHERE price between 2000 AND 5000;
```



OUTPUT

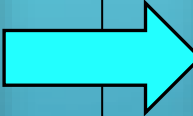
product_id	name	category	price	stock_quantity	added_on
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
13	Full West	Books	2112.33	172	2023-09-15 03:13:38
17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53

6. Fetch data for specific customer IDs (e.g., from loyalty program list)
This is used when customer IDs are pre-selected from another system.

INPUT

OUTPUT

```
SELECT * FROM customers  
where customer_id IN ("1","2","3","4","8");
```



customer_id	name	email	phone	created_at
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24
3	Kaitlin Richards	user3@example.com	2073473421	2024-06-23 09:55:22
4	Christina Williams	user4@example.com	586-605-5061x06	2024-10-27 17:19:38
8	John Bryan	user8@example.com	045.568.0798x27	2025-02-15 17:57:04

7. Identify customers whose names start with the letter 'A'.
Used for alphabetical segmentation in outreach or app display.

INPUT

```
select * FROM customers AS NAME_START_WITH_A  
WHERE NAME LIKE "A%";
```



OUTPUT

customer_id	name	email	phone
15	Austin Flores	user15@example.com	329.901.1576x66
16	Amy Landry	user16@example.com	+1-278-019-3748
19	Amanda Bright	user19@example.com	380.981.9798x69
27	Adrienne Green	user27@example.com	530.644.8455x93

8. List electronics products priced under ₹3,000

Used by merchandising or frontend teams to showcase budget electronics.

INPUT

```
SELECT NAME AS ELECTRONIC_PRODUCT_NAME, PRICE  
FROM products  
WHERE category="ELECTRONICS"  
AND price<3000;
```

OUTPUT



ELECTRONIC_PRODUCT_NAME	PRICE
Available Answer	2529.51
If Whatever	177.40
Place Low	723.97
Series Page	2070.37
Despite Win	1340.34
Actually Term	396.11
Southern Thing	512.46

9. Display product names and prices in descending order of price.
This helps teams easily view and compare top-priced items.

INPUT

```
SELECT NAME AS PRODUCT_NAME, PRICE AS PRODUCT_PRICE  
FROM products  
order by PRODUCT_PRICE DESC;
```



OUTPUT

PRODUCT_NAME	PRODUCT_PRICE
Response Indeed	4897.36
Population Social	4813.68
Development System	4801.78
Any Question	4759.28
Fire Often	4734.89
Natural Network	4722.66
Build High	4707.14
Serious Recognize	4523.10
Study Total	4413.68
Real Source	4398.66
Involve Officer	4244.09
Group But	4180.23

10. Display product names and prices, sorted by price and then by name

INPUT

```
SELECT NAME AS PRODUCT_NAME, PRICE AS PRODUCT_PRICE  
FROM products  
order by PRODUCT_PRICE, PRODUCT_NAME ;
```

OUTPUT

PRODUCT_NAME	PRODUCT_PRICE
If Whatever	177.40
Listen Development	296.17
Actually Term	396.11
Television Stock	421.73
Southern Thing	512.46
Plant No	639.43
Place Low	723.97
Stock Article	834.75
Toward Minute	857.85
Common Study	985.19
Answer But	1016.68
Special Fact	1094.18



LEVEL 2

1. Retrieve orders where customer information is missing (possibly due to data migration or deletion)
Used to identify orphaned orders or test data where Customer_ID is not linked.

INPUT

OUTPUT

```
SELECT * FROM ORDERS
LEFT JOIN customers
ON ORDERS.CUSTOMER_ID=CUSTOMERS.customer_id
where CUSTOMERS.customer_id IS NULL;
```

order_id	customer_id	order_date	status	total_amount	customer_id	name	email	phone	created_at
----------	-------------	------------	--------	--------------	-------------	------	-------	-------	------------

2. Display customer names and emails using column aliases for frontend readability
Useful for feeding into frontend displays or report headings that require user-friendly labels.

INPUT

```
SELECT NAME AS CUSTOMER_NAME, EMAIL AS CUSTOMER_EMAIL  
FROM customers;
```



OUTPUT

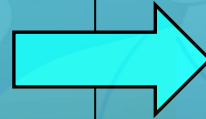
CUSTOMER_NAME	CUSTOMER_EMAIL
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Kaitlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com
Mark Duke	user6@example.com
Briana Wright	user7@example.com
John Bryan	user8@example.com
Jason Thompson	user9@example.com
Shawn Hill	user10@example.com
Walter Jenkins	user11@example.com
Mary Knight	user12@example.com

3. Calculate total value per item ordered by multiplying quantity and item price.

This can help generate per-line item bill details or invoice breakdowns.

INPUT

```
select order_item_id, sum(quantity*item_price)
as Total_Value_Order
from order_items
group by order_item_id;
```



OUTPUT

order_item_id	Total_Value
1	9414.28
2	532.20
3	2955.57
4	2208.99
5	9469.78
6	4707.14
7	9794.72
8	14692.08
9	1429.45
10	723.97

4. Combine customer name and phone number in a single column.
Used to show brief customer summaries or contact lists.

INPUT

```
select concat(NAME, " ", PHONE) AS Customers_phone  
FROM customers;
```



OUTPUT

Customers_phone	
Thomas Owens	142-479-1945
Charles Grant	9153947511
Kaitlin Richards	2073473421
Christina Williams	586-605-5061x06
David Allen	(751)456-8289x1
Mark Duke	(144)957-2811
Briana Wright	223-833-9635
John Bryan	045.568.0798x27
Jason Thompson	1862659420
Shawn Hill	(268)113-3152x7
Walter Jenkins	536-329-0817x71
Mary Knight	361-636-3802

5. Extract only the date part from order timestamps for date-wise reporting.

Helps group or filter orders by date without considering time.

INPUT

```
select order_id, date(order_date) as Order_date  
from orders;
```



OUTPUT

order_id	Order_date
1	2025-03-02
2	2024-10-09
3	2025-05-08
4	2024-09-19
5	2025-04-08
6	2024-10-25
7	2024-07-29
8	2024-07-30
9	2025-06-10
10	2025-02-16

6. List products that do not have any stock left.
This helps the inventory team identify out-of-stock items

INPUT

```
select product_id as product_name from products  
where stock_quantity=0;
```



OUTPUT

product_id	product_name
NULL	NULL

The background is a solid blue gradient. On the left side, there are white circuit-like lines and nodes. In the center, there is a faint illustration of a person with dark hair, wearing a blue shirt, sitting at a desk and using a laptop. Above the person's head, there is a faint icon of a database cylinder and a box labeled 'SQL'.

LEVEL 3

1. Count the total number of orders placed.
Used by business managers to track order volume over time.

INPUT

```
select count(*) as total_orders_placed  
from orders;
```



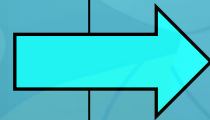
OUTPUT

total_orders_placed
400

2. Calculate the total revenue collected from all orders.
This gives the overall sales value.

INPUT

```
select sum(total_amount) as Total_Sales_Amount  
from orders  
where (status= 'Delivered');
```



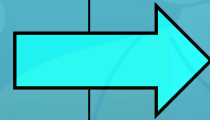
OUTPUT

Total_Sales_Amount
2022410.31

3. Calculate the average order value.
Used for understanding customer spending patterns.

INPUT

```
select round(avg(total_amount),2) as average_order_value  
from orders;
```



OUTPUT

average_order_value
17402.43

4. Count the number of customers who have placed at least one order.
This identifies active customers.

INPUT

```
select count(distinct(customer_id)) as customers_with_atleast_1_order  
from orders;
```



OUTPUT

customers_with_atleast_1_order
30

5. Find the number of orders placed by each customer.
Helpful for identifying top or repeat customers.

INPUT

```
select customer_id, count(order_id)
as total_order
from orders
group by customer_id;
```



OUTPUT

customer_id	total_order
1	12
2	17
3	17
4	9
5	14
6	16
7	13
8	10
9	10
10	13

6. Find total sales amount made by each customer

INPUT

```
select customer_id , sum(total_amount) as Total_Amount_Spend_By_Customers  
from Orders  
group by customer_id;
```



OUTPUT

customer_id	Total_Amount_Spend_By_Customers
1	183747.44
2	284420.07
3	253783.31
4	137562.22
5	262504.19
6	212173.58
7	167960.11
8	164701.78
9	106226.03
10	252722.75
11	173409.01
12	169653.79

7. List the number of products sold per category.
This helps category managers assess performance by department.

INPUT

```
select products.category ,  
count(order_items.quantity) as Count_Items  
from products  
join order_items  
on products.product_id=order_items.product_id  
group by products.category  
order by Count_Items desc;
```



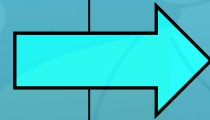
OUTPUT

category	Count_Items
Electronics	333
Clothing	281
Home	216
Toys	201
Books	170

8. Find the average item price per category.
Useful to compare pricing across departments.

INPUT

```
select category, round(avg(price),2) as average_item_price
from products
group by category
order by average_item_price;
```



OUTPUT

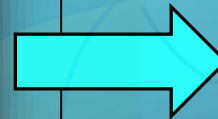
category	average_item_price
Home	2146.37
Toys	2516.53
Electronics	2653.39
Books	3167.08
Clothing	3434.58

9. Show number of orders placed per day.
Used to track daily business activity and demand trends.

INPUT

OUTPUT

```
select dayname(order_date) as Day_Name, count(order_id) as Total_Order
from orders
group by dayname(order_date)
order by total_order desc;
```



Day_Name	Total_Order
Friday	65
Wednesday	63
Monday	61
Tuesday	59
Thursday	51
Saturday	51
Sunday	50

10. List total payments received per payment method.
Helps the finance team understand preferred transaction modes.

INPUT

OUTPUT

```
select method as payment_mode, count(payment_id) as total_payment_Count
from payments
group by payment_mode
order by total_payment_Count desc;
```



payment_mode	total_payment_Count
Debit Card	113
Net Banking	98
Credit Card	96
UPI	93

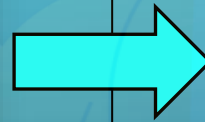


LEVEL 4

1. Retrieve order details along with the customer's name.
Used for displaying which customer placed each order.

INPUT

```
select customers.name as Customer_Name, orders.*  
from customers  
inner join orders  
on customers.customer_id= orders.customer_id;
```



OUTPUT

Customer_Name	order_id	customer_id	order_date	status	total_amount
Thomas Owens	14	1	2024-09-24 21:21:38	Shipped	15803.34
Thomas Owens	17	1	2024-08-19 21:17:57	Pending	11173.77
Thomas Owens	61	1	2024-12-26 23:21:58	Shipped	13053.00
Thomas Owens	76	1	2025-01-14 22:59:54	Pending	23506.81
Thomas Owens	92	1	2024-09-26 11:33:58	Shipped	834.75
Thomas Owens	109	1	2025-03-17 04:56:18	Shipped	12190.14
Thomas Owens	127	1	2025-06-10 18:01:32	Pending	20160.64
Thomas Owens	135	1	2025-03-11 22:58:23	Pending	8891.79
Thomas Owens	144	1	2024-09-19 09:18:22	Shipped	12093.54
Thomas Owens	221	1	2024-09-25 05:49:39	Pending	12437.61
Thomas Owens	278	1	2025-02-01 20:05:38	Delivered	32015.16
Thomas Owens	379	1	2025-03-22 15:57:58	Cancelled	21586.89

2. Get list of products that have been sold (INNER JOIN with order items).
Used to find which products were actually included in orders.

INPUT

```
select products.product_id,products.name  
from products  
inner join order_items  
on products.product_id = order_items.product_id  
group by product_id;
```



OUTPUT

product_id	name
1	Plant No
2	Population Social
3	Available Answer
4	Any Question
5	Natural Network
6	If Whatever
7	Response Indeed
8	Every Amount
9	Common Study
10	Development System

3. List all orders with their payment method (INNER JOIN)

INPUT

```
select orders.order_id,orders.order_date,orders.status,  
payments.method  
from orders  
inner join payments  
on orders.order_id = payments.order_id;
```



OUTPUT

order_id	order_date	status	method
1	2025-03-02 07:20:11	Delivered	Credit Card
2	2024-10-09 18:08:21	Shipped	Net Banking
3	2025-05-08 00:08:27	Cancelled	Credit Card
4	2024-09-19 22:16:13	Delivered	UPI
5	2025-04-08 18:02:06	Pending	UPI
6	2024-10-25 07:33:59	Cancelled	UPI
7	2024-07-29 11:58:47	Shipped	UPI
8	2024-07-30 22:49:49	Cancelled	Net Banking
9	2025-06-10 17:00:25	Pending	Net Banking
10	2025-02-16 12:45:59	Delivered	Debit Card

4. Get list of customers and their orders (LEFT JOIN).
Used to find all customers and see who has or hasn't placed orders.

INPUT

```
select customers.customer_id,orders.order_id,customers.name  
from customers  
left join orders  
on customers.customer_id = orders.customer_id;
```



OUTPUT

customer_id	order_id	name
1	14	Thomas Owens
1	17	Thomas Owens
1	61	Thomas Owens
1	76	Thomas Owens
1	92	Thomas Owens
1	109	Thomas Owens
1	127	Thomas Owens
1	135	Thomas Owens
1	144	Thomas Owens
1	221	Thomas Owens

5. List all products along with order item quantity (LEFT JOIN).
Useful for inventory teams to track what sold and what hasn't.

INPUT

```
select products.product_id,order_items.order_item_id,  
products.name,  
products.category,products.price,  
order_items.quantity  
from products  
left join order_items  
on products.product_id = order_items.product_id;
```



OUTPUT

product_id	order_item_id	name	category	price	quantity
1	20	Plant No	Home	639.43	1
1	23	Plant No	Home	639.43	3
1	72	Plant No	Home	639.43	1
1	103	Plant No	Home	639.43	1
1	140	Plant No	Home	639.43	2
1	210	Plant No	Home	639.43	3
1	223	Plant No	Home	639.43	2
1	248	Plant No	Home	639.43	2
1	372	Plant No	Home	639.43	2
1	398	Plant No	Home	639.43	2

6. List all payments including those with no matching orders (RIGHT JOIN).
Rare but used when ensuring all payments are mapped correctly

INPUT

```
select orders.order_id,payments.payment_id,  
payments.amount_paid,  
payments.payment_date  
from orders  
right join payments  
on orders.order_id = payments.order_id;
```



OUTPUT

order_id	payment_id	amount_paid	payment_date
1	1	9414.28	2025-03-02 08:16:11
2	2	532.20	2024-10-09 19:00:21
3	3	5164.56	2025-05-08 00:55:27
4	4	9469.78	2024-09-19 22:28:13
5	5	14501.86	2025-04-08 18:26:06
6	6	31050.17	2024-10-25 08:16:59
7	7	3043.67	2024-07-29 12:45:47
8	8	32714.06	2024-07-30 23:42:49
9	9	24219.20	2025-06-10 17:36:25
10	10	24342.52	2025-02-16 13:43:59

7. Combine data from three tables: customer, order, and payment.
Used for detailed transaction reports

INPUT

```
select c.customer_id,o.order_id,p.payment_id,  
c.name as customer_name, c.email,  
o.order_date,o.status,o.total_amount,  
p.payment_date,p.method  
from customers c  
join orders o  
on c.customer_id=o.customer_id  
join payments p  
on p.order_id = o.order_id;
```



OUTPUT

customer_id	order_id	payment_id	customer_name	email
1	14	14	Thomas Owens	user1@example.com
1	17	17	Thomas Owens	user1@example.com
1	61	61	Thomas Owens	user1@example.com
1	76	76	Thomas Owens	user1@example.com
1	92	92	Thomas Owens	user1@example.com
1	109	109	Thomas Owens	user1@example.com
1	127	127	Thomas Owens	user1@example.com
1	135	135	Thomas Owens	user1@example.com
1	144	144	Thomas Owens	user1@example.com
1	221	221	Thomas Owens	user1@example.com
1	278	278	Thomas Owens	user1@example.com

order_date	status	total_amount	payment_date	method
2024-09-24 21:21:38	Shipped	15803.34	2024-09-24 22:11:38	UPI
2024-08-19 21:17:57	Pending	11173.77	2024-08-19 21:49:57	UPI
2024-12-26 23:21:58	Shipped	13053.00	2024-12-26 23:31:58	Net Banking
2025-01-14 22:59:54	Pending	23506.81	2025-01-14 23:33:54	Credit Card
2024-09-26 11:33:58	Shipped	834.75	2024-09-26 12:03:58	Net Banking
2025-03-17 04:56:18	Shipped	12190.14	2025-03-17 05:16:18	UPI
2025-06-10 18:01:32	Pending	20160.64	2025-06-10 18:57:32	Debit Card
2025-03-11 22:58:23	Pending	8891.79	2025-03-11 23:11:23	Debit Card
2024-09-19 09:18:22	Shipped	12093.54	2024-09-19 10:10:22	UPI
2024-09-25 05:49:39	Pending	12437.61	2024-09-25 06:25:39	UPI
2025-02-01 20:05:38	Delivered	32015.16	2025-02-01 21:05:38	UPI

The background is a solid blue gradient. On the left side, there are white circuit-like lines and nodes. In the center, there is a faint illustration of a person with dark hair, wearing a blue shirt, sitting at a desk and using a laptop. Above the person's head, there is a faint box containing the text 'SQL'. To the left of the person, there are faint icons of a database cylinder, a gear, and a hierarchical tree structure.

LEVEL 5

1. List all products priced above the average product price.
Used by pricing analysts to identify premium-priced products.

INPUT

```
select name as Product_Name, price from products  
where price > (select avg(price) from products);
```



OUTPUT

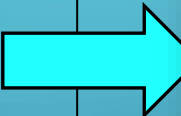
Product_Name	price
Population Social	4813.68
Any Question	4759.28
Natural Network	4722.66
Response Indeed	4897.36
Every Amount	4173.60
Development System	4801.78
Action Ask	4017.01
Some Them	3673.86
Build High	4707.14
Real Source	4398.66
Bed Including	3845.31
Move Small	4168.08

2. Find customers who have placed at least one order.
Used to identify active customers for loyalty campaigns.

INPUT

```
select *  
from customers  
where customer_id  
in (select distinct(customer_id)  
from orders where customer_id is not null);
```

OUTPUT



customer_id	name	email	phone	created_at
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24
3	Kaitlin Richards	user3@example.com	2073473421	2024-06-23 09:55:22
4	Christina Williams	user4@example.com	586-605-5061x06	2024-10-27 17:19:38
5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00
6	Mark Duke	user6@example.com	(144)957-2811	2024-06-24 03:22:59
7	Briana Wright	user7@example.com	223-833-9635	2023-06-25 00:35:43
8	John Bryan	user8@example.com	045.568.0798x27	2025-02-15 17:57:04
9	Jason Thompson	user9@example.com	1862659420	2024-08-31 08:18:51
10	Shawn Hill	user10@example.com	(268)113-3152x7	2023-12-14 20:46:43
11	Walter Jenkins	user11@example.com	536-329-0817x71	2023-10-26 03:12:30

3. Show orders whose total amount is above the average for that customer.
Used to detect unusually high purchases per customer.

INPUT

```
select o.customer_id,o.order_id,o.status,o.total_amount
from orders o
where o.total_amount>
(select avg(total_amount) from orders
where customer_id=o.customer_id);
```



OUTPUT

customer_id	order_id	status	total_amount
1	14	Shipped	15803.34
1	76	Pending	23506.81
1	127	Pending	20160.64
1	278	Delivered	32015.16
1	379	Cancelled	21586.89
2	223	Cancelled	41020.75
2	230	Pending	22655.32
2	315	Cancelled	42056.04

4. Display customers who haven't placed any orders.
Used for re-engagement campaigns targeting inactive users.

INPUT

```
select customer_id, name as customer_name  
from customers  
where customer_id not in  
(select customer_id from orders  
where customer_id is not null);
```



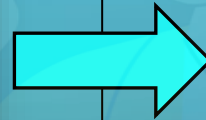
OUTPUT

customer_id	customer_name
HULL	HULL

5. Show products that were never ordered.
Helps with inventory clearance decisions or product deactivation.

INPUT

```
select product_id, name as product_name
from products
where product_id not in
(select product_id from order_items
where product_id is not null);
```



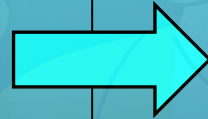
OUTPUT

product_id	product_name
NULL	NULL

6. Show highest value order per customer.
Used to identify the largest transaction made by each customer.

INPUT

```
select o.customer_id, o.order_id, o.total_amount
from orders o
where o.total_amount=
(select max(total_amount) from orders
where customer_id=o.customer_id)
order by o.customer_id asc;
```



OUTPUT

customer_id	order_id	total_amount
1	278	32015.16
2	315	42056.04
3	281	41679.11
4	386	25747.34
5	266	39921.78
6	330	39003.19
7	128	28589.04
8	33	36147.09
9	348	21414.44
10	62	35723.17

7. Highest Order Per Customer (Including Names).

Used to identify the largest transaction made by each customer. Outputs name as well.

INPUT

```
select c.customer_id,o.order_id,c.name as customer_name,  
o.total_amount as Highest_Value_order  
from customers c  
join orders o  
on c.customer_id=o.customer_id  
where o.total_amount=(select max(total_amount)
```



OUTPUT

customer_id	order_id	customer_name	Highest_Value_order
1	278	Thomas Owens	32015.16
2	315	Charles Grant	42056.04
3	281	Kaitlin Richards	41679.11
4	386	Christina Williams	25747.34
5	266	David Allen	39921.78
6	330	Mark Duke	39003.19
7	128	Briana Wright	28589.04
8	33	John Bryan	36147.09
9	348	Jason Thompson	21414.44
10	62	Shawn Hill	35723.17



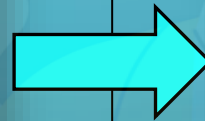
LEVEL 6

1. List all customers who have either placed an order or written a product review.

Used to identify engaged customers from different activity areas.

INPUT

```
select * from customers where customer_id in
(select customer_id from orders
union
select customer_id from product_reviews
where customer_id is not null
);
```



OUTPUT

customer_id	name	email	phone	created_at
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24
3	Kaitlin Richards	user3@example.com	2073473421	2024-06-23 09:55:22
4	Christina Williams	user4@example.com	586-605-5061x06	2024-10-27 17:19:38
5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00
6	Mark Duke	user6@example.com	(144)957-2811	2024-06-24 03:22:59
7	Briana Wright	user7@example.com	223-833-9635	2023-06-25 00:35:43
8	John Bryan	user8@example.com	045.568.0798x27	2025-02-15 17:57:04
9	Jason Thompson	user9@example.com	1862659420	2024-08-31 08:18:51
10	Shawn Hill	user10@example.com	(268)113-3152x7	2023-12-14 20:46:43

2. List all customers who have placed an order as well as reviewed a product [intersect not supported].

Used to identify highly engaged customers for rewards.

INPUT

```
select * from customers where customer_id in
(select customer_id from orders)
and
customer_id in
(select customer_id from product_reviews
where customer_id is not null);
```



OUTPUT

customer_id	name	email	phone	created_at
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24
4	Christina Williams	user4@example.com	586-605-5061x06	2024-10-27 17:19:38
6	Mark Duke	user6@example.com	(144)957-2811	2024-06-24 03:22:59
7	Briana Wright	user7@example.com	223-833-9635	2023-06-25 00:35:43
9	Jason Thompson	user9@example.com	1862659420	2024-08-31 08:18:51
10	Shawn Hill	user10@example.com	(268)113-3152x7	2023-12-14 20:46:43
11	Walter Jenkins	user11@example.com	536-329-0817x71	2023-10-26 03:12:30
13	Leslie Wilson	user13@example.com	+1-256-261-1984	2024-06-06 20:12:35
14	Deborah Arias	user14@example.com	811-821-2144x97	2024-04-24 00:27:28

The background is a solid blue gradient. On the left side, there are white circuit-like lines and circles. In the center, there is a faint illustration of a person with dark hair sitting at a desk with a laptop. Above the person's head, there is a faint box containing the text 'SQL'.

THE END